Introduce Server side and Client Side Scripting

1. Client side scripting

web browsers execute client side scripting. It is use when browsers has all code. Source code used to transfer from web server to user's computer over internet and run directly on browsers. It is also used for validations and functionality for user events.

- It allows for more interactivity.
- It usually performs several actions without going to user.
- It cannot be basically used to connect to databases on web server.
- These scripts cannot access file system that resides at web browser. Pages are altered on basis of users choice. It can also used to create "cookies" that store data on user's computer.

2. Server side scripting

Web servers are used to execute server side scripting. They are basically used to create dynamic pages. It can also access the file system residing at web server. Server-side environment that runs on a scripting language is a web-server.

- Scripts can be written in any of a number of server-side scripting language available. It is used to retrieve and generate content for dynamic pages.
- It is used to require to download plugins. In this load times are generally faster than client-side scripting.
- When you need to store and retrieve information a database will be used to contain data. It can use huge resources of server.
- It reduces client-side computation overhead. Server sends pages to request of user/client.

Client side scripting	Server side scripting
1.Source code is visible to user.	**1**.Source code is not visible to user because it's output of server side is a HTML page.
2.It usually depends on browser and it's version.	**2**.In this any server side technology can be use and it does not depend on client.
3.It runs on user's computer.	**3**.It runs on web server.
4.There are many advantages link with this like faster. response times, a more interactive application.	**4.**The primary advantage is it's ability to highly customize, response requirements, access rights based on user.
5.It does not provide security for data.	**5.**It provides more security for data.
6.It is a technique use in web development in which scripts runs on clients browser.	**6.**It is a technique that uses scripts on web server to produce a response that is customized for each clients request.
7.HTML, CSS and javascript are	**7.**PHP, Python, Java, Ruby are used.

used.	

Server Side Scripting using PHP

PHP is a server-side scripting language designed to be used for web purposes.Originally PHP was short for "Personal Home Page" but over time it evolved to include that in its recursive current expansion "PHP: Hypertext Preprocessor".

- Server-side scripting languages interpret scripts on the server side rather than client-side (like JavaScript).
- Doing so provides a customized interface for each user and adds functionality beyond what HTML can offer.Scripting languages are programming languages that are interpreted rather than needing to be compiled before execution.

Advantages of PHP

- Open Source
- Inexpensive hosting and setup
- Cross-Platform
- Client applications don't need PHP installed
- Lots of free libraries and packages available for use

Disadvantages of PHP

- PHP's main strength - flexibility - is also its weakness. It can be a little too forgiving of errors.
- With no strict rules, inexperienced programmers have the freedom to create some very bad solutions to simple problems.
- Bad packages that got popular in the community are still reused when developers are trying something new or rushed for time. Some of these mistakes lead to security risks.
- As a mature tool PHP has some legacy baggage. There are lots of internal consistencies, especially surrounding references and values.
- This is mostly due to updates which add features that clash with earlier features.
- PHP is an interpreted language, which can reduce speed.
- PHP 7 increased performance over previous versions by a significant amount while maintaining most language compatibility.

.

Applications

- Current users of PHP include Joomla, Wikipedia, Yahoo!, Tumblr, and Slack.
- Facebook liked it so much that, instead of migrating away as they grew, they created a custom PHP-derived language called Hack to let them continue using it at scale.
- Facebook might be the largest single company that features PHP, but WordPress is a major driver of its continued usage.

- It's written using PHP as the scripting language. While the average customer doesn't need to know PHP to set up a site, anyone who makes WordPress themes or plugins needs at least a basic understanding of the language.
- WordPress powers nearly a third of all published sites and is the fastest growing CMS available, so PHP can be found in most corners of the internet.

.

Object oriented programming with server side scripting(php)

Object Oriented Concepts

Before we go in detail, lets define important terms related to Object Oriented Programming.

- **Class** − This is a programmer-defined data type, which includes local functions as well as local data. You can think of a class as a template for making many instances of the same kind (or class) of object.
- **Object** − An individual instance of the data structure defined by a class. You define a class once and then make many objects that belong to it. Objects are also known as instance.
- **Member Variable** − These are the variables defined inside a class. This data will be invisible to the outside of the class and can be accessed via member functions. These variables are called attribute of the object once an object is created.
- **Member function** − These are the function defined inside a class and are used to access object data.
- **Inheritance** − When a class is defined by inheriting existing function of a parent class then it is called inheritance. Here child class will inherit all or few member functions and variables of a parent class.
- **Parent class** − A class that is inherited from by another class. This is also called a base class or super class.
- **Child Class** − A class that inherits from another class. This is also called a subclass or derived class.
- **Polymorphism** − This is an object oriented concept where same function can be used for different purposes. For example function name will remain same but it take different number of arguments and can do different task.
- **Overloading** − a type of polymorphism in which some or all of operators have different implementations depending on the types of their arguments. Similarly functions can also be overloaded with different implementation.
- **Data Abstraction** − Any representation of data in which the implementation details are hidden (abstracted).
- **Encapsulation** − refers to a concept where we encapsulate all the data and member functions together to form an object.
- **Constructor** − refers to a special type of function which will be called automatically whenever there is an object formation from a class.

- **Destructor** – refers to a special type of function which will be called automatically whenever an object is deleted or goes out of scope.

3.15 Basic PHP syntax

The PHP script is executed on the server and the HTML result is sent to the browser. It can normally have HTML and PHP tags. PHP or Hypertext Preprocessor is a widely used open-source general-purpose scripting language and can be embedded with HTML. PHP files are saved with the ".php" extension. PHP scripts can be written anywhere in the document within PHP tags along with normal HTML.

Canonical PHP Tags: The script starts with **<?php** and ends with **?>**. These tags are also called 'Canonical PHP tags'. Everything outside of a pair of opening and closing tags is ignored by the PHP parser. The open and closing tags are called delimiters. Every PHP command ends with a semi-colon (**;**).

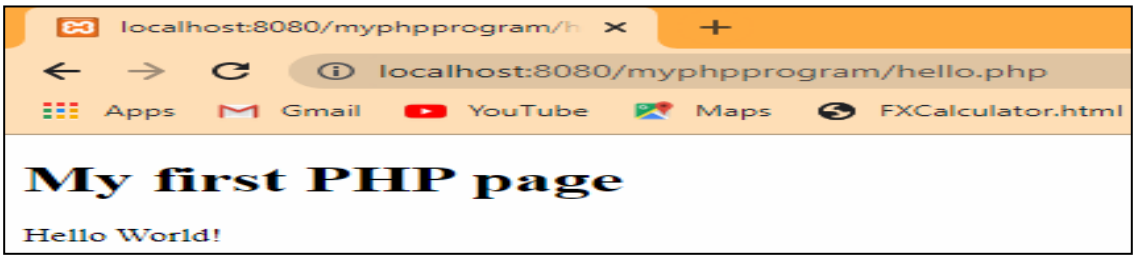
Let's look at the *hello world* program in PHP.

```
<!DOCTYPE html>
<html>
<body>

<h1>My first PHP page</h1>

<?php
echo "Hello World!";
?>
</body>
</html>
```

Output:



Lexical Structure

The lexical structure of a programming language is the set of basic rules that governs how you write programs in that language. It is the lowest-level syntax of the language and specifies such things as what variable names look like, what characters are used for comments, and how program statements are separated from each other.

Case Sensitivity

The names of user-defined classes and functions, as well as built-in constructs and keywords (such as echo, while, class, etc.), are case-insensitive. Thus, these three lines are equivalent:

```
echo("hello, world");
ECHO("hello, world");
EcHo("hello, world");
```

Variables, on the other hand, are case-sensitive. That is, $name, $NAME, and $NaME are three different variables.

Statements and Semicolons

A statement is a collection of PHP code that does something. It can be as simple as a variable assignment or as complicated as a loop with multiple exit points. Here is a small sample of PHP statements, including function calls, some variable data assign ments, and an if statement:

```
echo "Hello, world";
myFunction(42, "O'Reilly");
$a = 1;
$name = "Elphaba";
$b = $a / 25.0;
if ($a == $b) {
echo "Rhyme? And Reason?";
}
```

PHP uses semicolons to separate simple statements. A compound statement that uses curly braces to mark a block of code, such as a conditional test or loop, does not need a semicolon after a closing brace. Unlike in other languages, in PHP the semicolon before the closing brace is not optional:

```
if ($needed) {
echo "We must have it!"; // semicolon required here
```

```
} // no semicolon required here after the brace
```

The semicolon, however, is optional before a closing PHP tag:

```
<?php
if ($a == $b) {
echo "Rhyme? And Reason?";
}
echo "Hello, world" // no semicolon required before closing tag
?>
```

It's good programming practice to include optional semicolons, as they make it easier to add code later.

Whitespace and Line Breaks

In general, whitespace doesn't matter in a PHP program. You can spread a statement across any number of lines, or lump a bunch of statements together on a single line. For example, this statement:

```
raisePrices($inventory, $inflation, $costOfLiving, $greed);
```

could just as well be written with more whitespace:

```
raisePrices (
$inventory ,
$inflation ,
$costOfLiving ,
$greed
) ;
```

or with less whitespace:

```
raisePrices($inventory,$inflation,$costOfLiving,$greed);
```

You can take advantage of this flexible formatting to make your code more readable (by lining up assignments, indenting, etc.). Some lazy programmers take advantage of this freeform formatting and create completely unreadable code—this is not recommended.

Comments

Comments give information to people who read your code, but they are ignored by PHP at execution time. Even if you think you're the only person who will ever read your code, it's a good idea to include comments in your code—in retrospect, code you wrote months ago could easily look as though a stranger wrote it.

A good practice is to make your comments sparse enough not to get in the way of the code itself but plentiful enough that you can use the comments to tell what's happen ing. Don't comment obvious things, lest you bury the comments that describe tricky things. For example, this is worthless:

```
$x = 17; // store 17 into the variable $x
```

whereas the comments on this complex regular expression will help whoever main tains your code:

```
// convert &#nnn; entities into characters
$text = preg_replace('/&#([0-9])+;/', "chr('\\1')", $text);
```

PHP provides several ways to include comments within your code, all of which are borrowed from existing languages such as C, C++, and the Unix shell. In general, use C-style comments to comment out code, and C++-style comments to comment on code.

Shell-style comments

When PHP encounters a hash mark character (#) within the code, everything from the hash mark to the end of the line or the end of the section of PHP code (whichever comes first) is considered a comment. This method of commenting is found in Unix shell scripting languages and is useful for annotating single lines of code or making short notes.

Because the hash mark is visible on the page, shell-style comments are sometimes used to mark off blocks of code:

```
#######################
## Cookie functions
#######################
```

Sometimes they're used before a line of code to identify what that code does, in which case they're usually indented to the same level as the code for which the comment is intended:

```
if ($doubleCheck) {
# create an HTML form requesting that the user confirm the action
echo confirmationForm();
}
```

Short comments on a single line of code are often put on the same line as the code:

```
$value = $p * exp($r * $t); # calculate compounded interest
```

When you're tightly mixing HTML and PHP code, it can be useful to have the closing PHP tag terminate the comment:

```
<?php $d = 4; # Set $d to 4. ?> Then another <?php echo $d; ?>
```

Then another 4

C++ comments

When PHP encounters two slashes (//) within the code, everything from the slashes to the end of the line or the end of the section of code, whichever comes first, is considered a comment. This method of commenting is derived from C++. The result is the same as the shell comment style.

Here are the shell-style comment examples, rewritten to use C++ comments:

```
//////////////////////
// Cookie functions
//////////////////////
if ($doubleCheck) {
// create an HTML form requesting that the user confirm the action
echo confirmationForm();
}
$value = $p * exp($r * $t); // calculate compounded interest
<?php $d = 4; // Set $d to 4. ?> Then another <?php echo $d; ?>
```

Then another 4

C comments

While shell-style and C++-style comments are useful for annotating code or making short notes, longer comments require a different style. Therefore, PHP supports block comments whose syntax comes from the C programming language. When PHP encounters a slash followed by an asterisk (/*), everything after that, until it encounters an asterisk followed by a slash (*/), is considered a comment. This kind of comment, unlike those shown earlier, can span multiple lines.

Here's an example of a C-style multiline comment:

```
/* In this section, we take a bunch of variables and
 assign numbers to them. There is no real reason to
 do this, we're just having fun.
*/
$a = 1;
```

```
$b = 2;
$c = 3;
$d = 4;
```

Because C-style comments have specific start and end markers, you can tightly inte grate them with code. This tends to make your code harder to read and is discouraged:

```
/* These comments can be mixed with code too,
see? */ $e = 5; /* This works just fine. */
```

C-style comments, unlike the other types, can continue past the end PHP tag mark ers. For example:

```
<?php
$l = 12;
$m = 13;
/* A comment begins here
?>
<p>Some stuff you want to be HTML.</p>
<?= $n = 14; ?>
*/
echo("l=$l m=$m n=$n\n");
?><p>Now <b>this</b> is regular HTML...</p>
l=12 m=13 n=
<p>Now <b>this</b> is regular HTML...</p>
```

You can indent comments as you like:

```
/* There are no
 special indenting or spacing
 rules that have to be followed, either.
 */
```

C-style comments can be useful for disabling sections of code. In the following exam ple, we've disabled the second and third statements, as well as the inline comment, by including them in a block comment. To enable the code, all we have to do is remove the comment markers:

```
$f = 6;
/*
```

$g = 7; # This is a different style of comment

$h = 8;

***/**

However, you have to be careful not to attempt to nest block comments:

$i = 9;

/*

$j = 10; /* This is a comment */

$k = 11;

Here is some comment text.

***/**

In this case, PHP tries (and fails) to execute the (non)statement Here is some comment text and returns an error.

Literals

A literal is a data value that appears directly in a program. The following are all literals in PHP:

2001

0xFE

1.4142

"Hello World"

'Hi'

true

null

Identifiers

An identifier is simply a name. In PHP, identifiers are used to name variables, functions, constants, and classes. The first character of an identifier must be an ASCII letter (uppercase or lowercase), the underscore character (_), or any of the characters between ASCII 0x7F and ASCII 0xFF. After the initial character, these characters and the digits 0–9 are valid.

Variable names

Variable names always begin with a dollar sign ($) and are case-sensitive. Here are some valid variable names:

$bill

$head_count

$MaximumForce

$I_HEART_PHP

$_underscore

$_int

Here are some illegal variable names:

$not valid

$|

$3wa

These variables are all different due to case sensitivity:

$hot_stuff $Hot_stuff $hot_Stuff $HOT_STUFF

Function names

Function names are not case-sensitive (functions are discussed in more detail in Chapter 3). Here are some valid function names:

tally

list_all_users

deleteTclFiles

LOWERCASE_IS_FOR_WIMPS

_hide

These function names all refer to the same function:

howdy HoWdY HOWDY HOWdy howdy

Class names

Class names follow the standard rules for PHP identifiers and are also not casesensitive. Here are some valid class names:

Person

account

The class name stdClass is a reserved class name.

Constants

A constant is an identifier for a value that will not be changed; scalar values (Boolean, integer, double, and string) and arrays can be constants. Once set, the value of a con stant cannot change. Constants are referred to by their identifiers and are set using the define() function:

```
define('PUBLISHER', "O'Reilly Media");
echo PUBLISHER;
```

Keywords

A keyword (or reserved word) is a word set aside by the language for its core func tionality—you cannot give a function, class, or constant the same name as a keyword. Table 2-1 lists the keywords in PHP, which are case-insensitive.

Table 2-1. PHP core language keywords

__CLASS__	elseif	new
__DIR__	empty()	or
__FILE__	enddeclare	print
__FUNCTION__	endfor	private
__LINE__	endforeach	protected
__METHOD__	endif	public
__NAMESPACE__	endswitch	require
__TRAIT__	endwhile	require_once
__halt_compiler()	eval()	return
abstract	exit()	static
and	extends	switch
array()	final	throw
as	finally	trait
break	for	try
callable	foreach	unset()
case	function	use
catch	global	var
class	goto	while
clone	if	xor
const	implements	yield
continue	include	yield from
declare	include_once	
default	instanceof	
die()	insteadof	
do	interface	
echo	isset()	
else	list()	
	namespace	

PHP Data Types

PHP data types are used to hold different types of data or values. PHP supports 8 primitive data types that can be categorized further in 3 types:

- ✓ Scalar Types (predefined)
- ✓ Compound Types (user-defined)
- ✓ Special Types

✓ PHP Data Types: Scalar Types

It holds only single value. There are 4 scalar data types in PHP.

- ❖ boolean
- ❖ integer
- ❖ float
- ❖ string

PHP Data Types: Compound Types

It can hold multiple values. There are 2 compound data types in PHP.

- ❖ array
- ❖ object

PHP Data Types: Special Types

There are 2 special data types in PHP.

- ❖ resource
- ❖ NULL

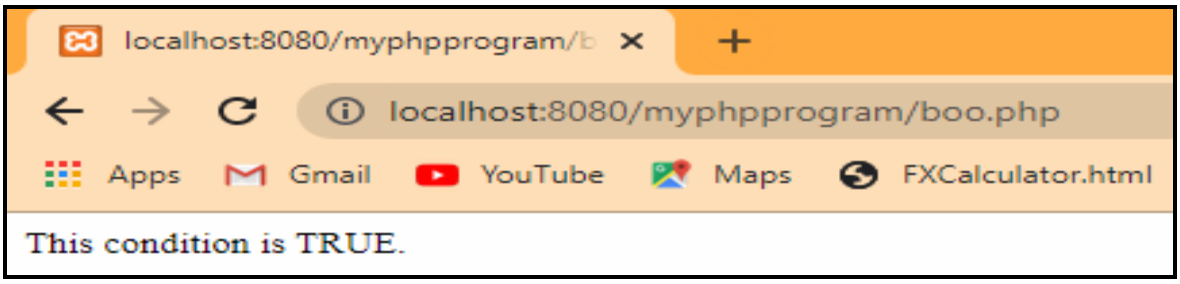
PHP Boolean

Booleans are the simplest data type works like switch. It holds only two values: TRUE (1) or FALSE (0). It is often used with conditional statements. If the condition is correct, it returns TRUE otherwise FALSE.

Example:

```
<?php
   if (TRUE)
      echo "This condition is TRUE.";
   if (FALSE)
      echo "This condition is FALSE.";
?>
```

Output:



PHP Integer

Integer means numeric data with a negative or positive sign. It holds only whole numbers, i.e., numbers without fractional part or decimal points.

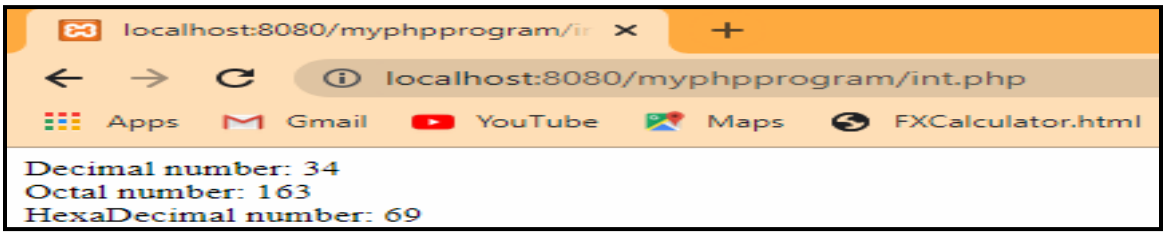
Rules for integer:

- An integer can be either positive or negative.
- An integer must not contain decimal point.
- Integer can be decimal (base 10), octal (base 8), or hexadecimal (base 16).
- The range of an integer must be lie between 2,147,483,648 and 2,147,483,647 i.e., -2^31 to 2^31.

Example:

```
<?php
   $dec1 = 34;
   $oct1 = 0243;
   $hexa1 = 0x45;
   echo "Decimal number: " .$dec1. "</br>";
   echo "Octal number: " .$oct1. "</br>";
   echo "HexaDecimal number: " .$hexa1. "</br>";
?>
```

Output:



PHP Float

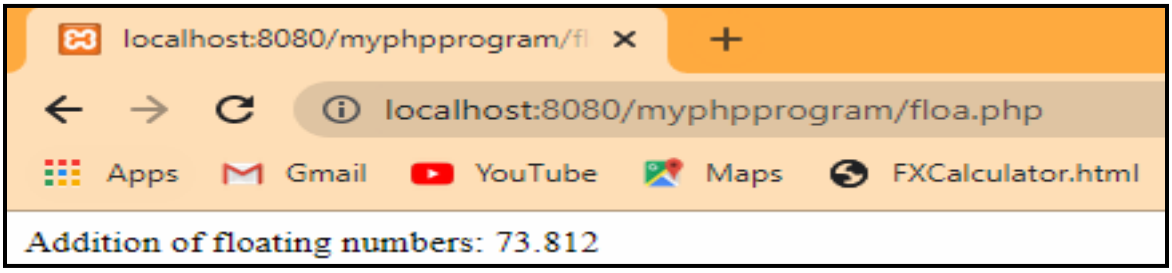
A floating-point number is a number with a decimal point. Unlike integer, it can hold numbers with a fractional or decimal point, including a negative or positive sign.

Example:

```
<?php
```

```
    $n1 = 19.34;
    $n2 = 54.472;
    $sum = $n1 + $n2;
    echo "Addition of floating numbers: " .$sum;
?>
```

Output:



PHP String

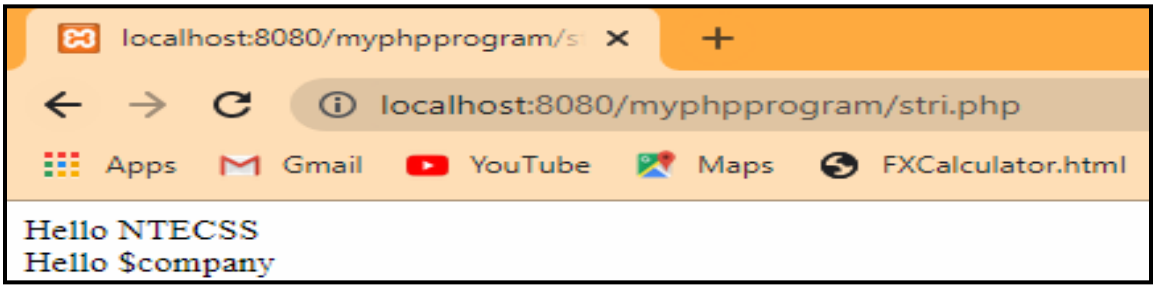
A string is a non-numeric data type. It holds letters or any alphabets, numbers, and even special characters.

String values must be enclosed either within single quotes or in double quotes. But both are treated differently. To clarify this, see the example below:

Example:

```
<?php
    $company = "NTECSS";
    //both single and double quote statements will treat different
    echo "Hello $company";
    echo "</br>";
    echo 'Hello $company';
?>
```

Output:



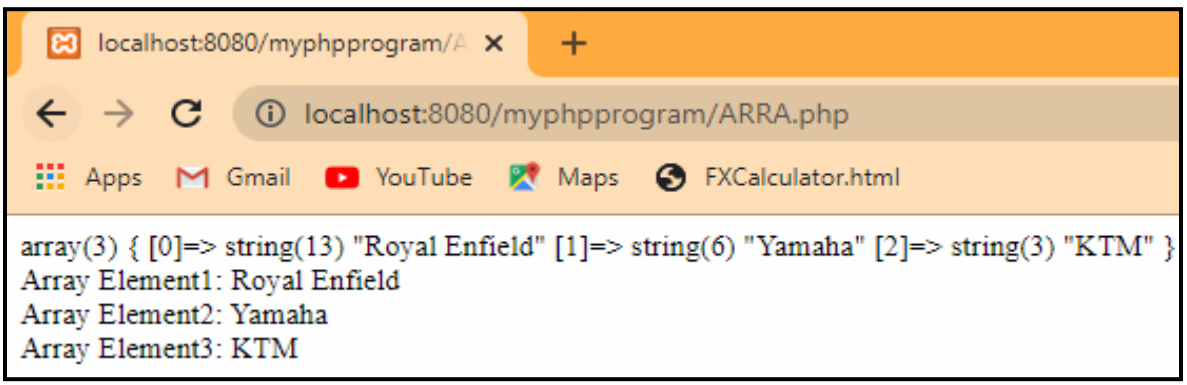
PHP Array

An array is a compound data type. It can store multiple values of same data type in a single variable.

Example:

```
<?php
   $bikes = array ("Royal Enfield", "Yamaha", "KTM");
   var_dump($bikes);   //the var_dump() function returns the datatype and v
alues
   echo "</br>";
   echo "Array Element1: $bikes[0] </br>";
   echo "Array Element2: $bikes[1] </br>";
   echo "Array Element3: $bikes[2] </br>";
?>
```

Output:



PHP object

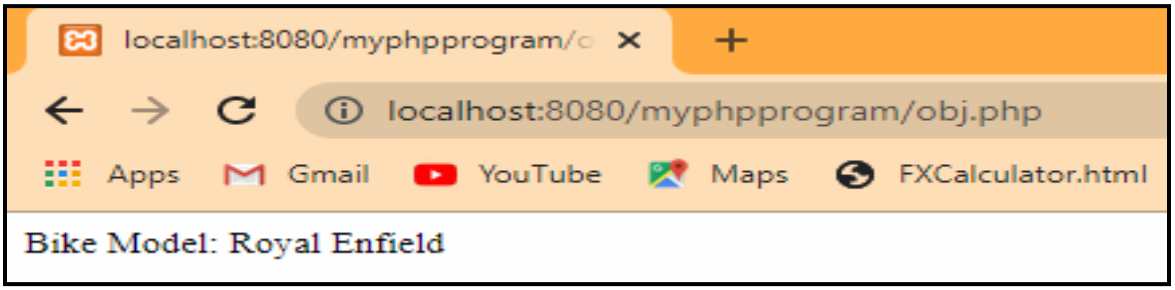
Objects are the instances of user-defined classes that can store both values and functions. They must be explicitly declared.

Example:

```
<?php
    class bike {
        function model() {
            $model_name = "Royal Enfield";
            echo "Bike Model: " .$model_name;
          }
    }
    $obj = new bike();
    $obj -> model();
```

```
?>
```

Output:



PHP Null

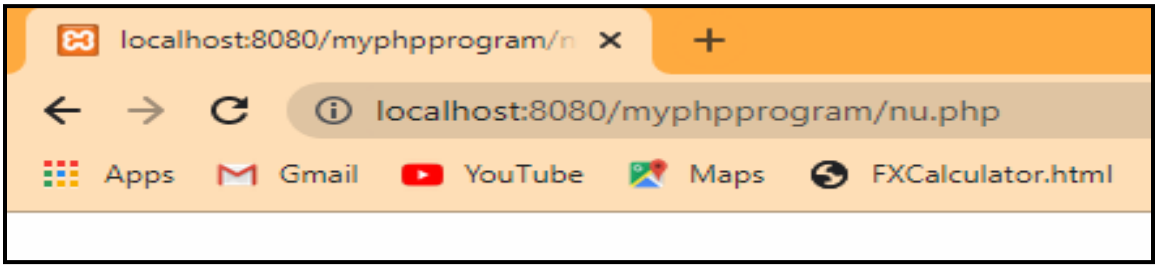
Null is a special data type that has only one value: NULL. There is a convention of writing it in capital letters as it is case sensitive.

The special type of data type NULL defined a variable with no value.

Example:

```
<?php
   $nl = NULL;
   echo $nl;   //it will not give any output
?>
```

Output:



PHP Variables

In PHP, a variable is declared using a **$ sign** followed by the variable name. Here, some important points to know about variables:

- As PHP is a loosely typed language, so we do not need to declare the data types of the variables. It automatically analyzes the values and makes conversions to its correct datatype.

- After declaring a variable, it can be reused throughout the code.
- Assignment Operator (=) is used to assign the value to a variable.

Syntax of declaring a variable in PHP is given below:

$variablename=value;

Rules for declaring PHP variable:

- A variable must start with a dollar ($) sign, followed by the variable name.
- It can only contain alpha-numeric character and underscore (A-z, 0-9, _).
- A variable name must start with a letter or underscore (_) character.
- A PHP variable name cannot contain spaces.
- One thing to be kept in mind that the variable name cannot start with a number or special symbols.
- PHP variables are case-sensitive, so $name and $NAME both are treated as different variable.

PHP Variable: Declaring string, integer, and float

Let's see the example to store string, integer, and float values in PHP variables.

File: variable1.php

```
<?php
$str="hello string";
$x=200;
$y=44.6;
echo "string is: $str <br/>";
echo "integer is: $x <br/>";
echo "float is: $y <br/>";
?>
```

Output:

```
string is: hello string
integer is: 200
float is: 44.6
```

PHP Variable: case sensitive

In PHP, variable names are case sensitive. So variable name "color" is different from Color, COLOR, COLor etc.

File: variable3.php

```
<?php
$color="red";
echo "My car is " . $color . "<br>";
echo "My house is " . $COLOR . "<br>";
echo "My boat is " . $coLOR . "<br>";
?>
```

Output:

```
My car is red
Notice: Undefined variable: COLOR in C:\wamp\www\variable.php on line 4
My house is
Notice: Undefined variable: coLOR in C:\wamp\www\variable.php on line 5
My boat is
```

PHP Variable: Rules

PHP variables must start with letter or underscore only.

PHP variable can't be start with numbers and special symbols.

File: variablevalid.php

```
<?php
$a="hello";//letter (valid)
$_b="hello";//underscore (valid)

echo "$a <br/> $_b";
?>
```

Output:

```
hello
hello
```

File: variableinvalid.php

```
1. <?php
2. $4c="hello";//number (invalid)
3. $*d="hello";//special symbol (invalid)
4.
5. echo "$4c <br/> $*d";
6. ?>
```

Output:

```
Parse error: syntax error, unexpected '4' (T_LNUMBER), expecting variable (T_VARIABLE)
 or '$' in C:\wamp\www\variableinvalid.php on line 2
```

PHP: Loosely typed language

PHP is a loosely typed language, it means PHP automatically converts the variable to its correct data type.

PHP Variable Scope

The scope of a variable is defined as its range in the program under which it can be accessed. In other words, "The scope of a variable is the portion of the program within which it is defined and can be accessed."

PHP has three types of variable scopes:

1. Local variable
2. Global variable
3. Static variable

Local variable

The variables that are declared within a function are called local variables for that function. These local variables have their scope only in that particular function in which they are declared. This means that these variables cannot be accessed outside the function, as they have local scope.

A variable declaration outside the function with the same name is completely different from the variable declared inside the function. Let's understand the local variables with the help of an example:

File: local_variable1.php

```
1. <?php
2.    function local_var()
3.    {
4.        $num = 45;  //local variable
5.        echo "Local variable declared inside the function is: ". $num;
6.    }
7.    local_var();
8. ?>
```

Output:

```
Local variable declared inside the function is: 45
```

File: local_variable2.php

```
1. <?php
2.    function mytest()
3.    {
4.        $lang = "PHP";
5.        echo "Web development language: " .$lang;
6.    }
7.    mytest();
8.    //using $lang (local variable) outside the function will generate an error
9.    echo $lang;
10. ?>
```

Output:

```
Web development language: PHP
Notice: Undefined variable: lang in D:\xampp\htdocs\program\p3.php on line 28
```

Global variable

The global variables are the variables that are declared outside the function. These variables can be accessed anywhere in the program. To access the global variable within a function, use the GLOBAL keyword before the variable. However, these variables can be directly accessed or used outside the function without any keyword. Therefore there is no need to use any keyword to access a global variable outside the function.

Let's understand the global variables with the help of an example:

Example:

File: global_variable1.php

```
1. <?php
2.    $name = "Sanaya Sharma";       //Global Variable
3.    function global_var()
4.    {
5.       global $name;
6.       echo "Variable inside the function: ". $name;
7.       echo "</br>";
8.    }
9.    global_var();
10.   echo "Variable outside the function: ". $name;
11. ?>
```

Output:

```
Variable inside the function: Sanaya Sharma
Variable outside the function: Sanaya Sharma
```

> Note: Without using the global keyword, if you try to access a global variable inside the function, it will generate an error that the variable is undefined.

Example:

File: global_variable2.php

```
1. <?php
2.    $name = "Sanaya Sharma";       //global variable
3.    function global_var()
4.    {
5.       echo "Variable inside the function: ". $name;
6.       echo "</br>";
7.    }
8.    global_var();
9. ?>
```

Output:

```
Notice: Undefined variable: name in D:\xampp\htdocs\program\p3.php on line 6
Variable inside the function:
```

Using $GLOBALS instead of global

Another way to use the global variable inside the function is predefined $GLOBALS array.

Example:

File: global_variable3.php

```
1. <?php
2.    $num1 = 5;      //global variable
3.    $num2 = 13;     //global variable
4.    function global_var()
5.    {
6.          $sum = $GLOBALS['num1'] + $GLOBALS['num2'];
7.          echo "Sum of global variables is: " .$sum;
8.    }
9.    global_var();
10. ?>
```

Output:

```
Sum of global variables is: 18
```

If two variables, local and global, have the same name, then the local variable has higher priority than the global variable inside the function.

Example:

File: global_variable2.php

```
1. <?php
2.    $x = 5;
3.    function mytest()
4.    {
5.       $x = 7;
6.       echo "value of x: " .$x;
7.    }
8.    mytest();
9. ?>
```

Output:

```
Value of x: 7
```

Note: local variable has higher priority than the global variable.

Static variable

It is a feature of PHP to delete the variable, once it completes its execution and memory is freed. Sometimes we need to store a variable even after completion of function execution. Therefore, another important feature of variable scoping is static variable. We use the static keyword before the variable to define a variable, and this variable is called as **static variable**.

Static variables exist only in a local function, but it does not free its memory after the program execution leaves the scope. Understand it with the help of an example:

Example:

File: static_variable.php

```
1. <?php
2.     function static_var()
3.     {
4.         static $num1 = 3;       //static variable
5.         $num2 = 6;          //Non-static variable
6.         //increment in non-static variable
7.         $num1++;
8.         //increment in static variable
9.         $num2++;
10.        echo "Static: " .$num1 ."</br>";
11.        echo "Non-static: " .$num2 ."</br>";
12.    }
13.
14. //first function call
15.    static_var();
16.
17.    //second function call
18.    static_var();
19. ?>
```

Output:

```
Static: 4
Non-static: 7
Static: 5
Non-static: 7
```

You have to notice that $num1 regularly increments after each function call, whereas $num2 does not. This is why because $num1 is not a static variable, so it freed its memory after the execution of each function call.

PHP $ and $$ Variables

The **$var** (single dollar) is a normal variable with the name var that stores any value like string, integer, float, etc.

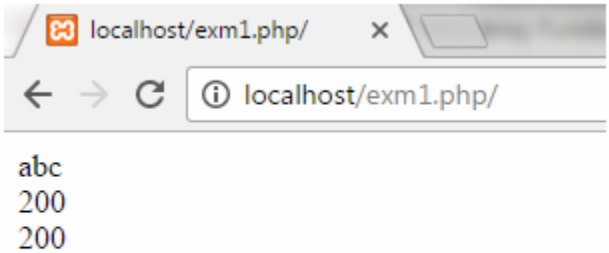
The **$$var** (double dollar) is a reference variable that stores the value of the $variable inside it.

To understand the difference better, let's see some examples.

Example 1

```
<?php
1. $x = "abc";
2. $$x = 200;
3. echo $x."<br/>";
4. echo $$x."<br/>";
5. echo $abc;
6. ?>
```

Output:



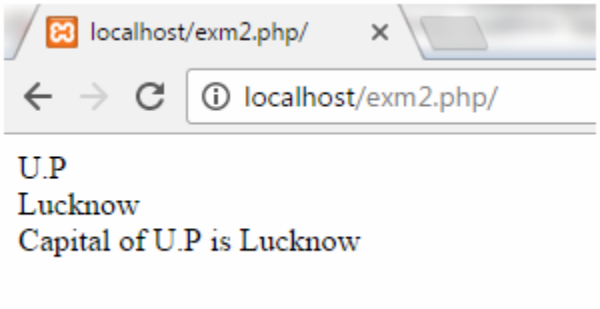
In the above example, we have assigned a value to the variable **x** as **abc**. Value of reference variable **$$x** is assigned as **200**.

Now we have printed the values **$x, $$x** and **$abc**.

Example2

```
1. <?php
2.  $x="U.P";
3. $$x="Lucknow";
4. echo $x. "<br>";
5. echo $$x. "<br>";
6. echo "Capital of $x is " . $$x;
7. ?>
```

Output:



In the above example, we have assigned a value to the variable **x** as **U.P**. Value of reference variable **$$x** is assigned as **Lucknow.**

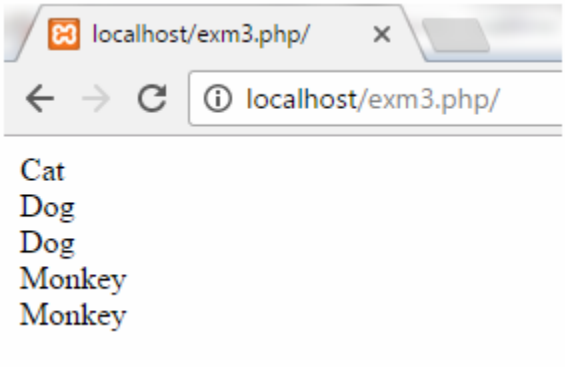
Now we have printed the values **$x, $$x** and a string.

Example3

```
1. <?php
2. $name="Cat";
3. ${$name}="Dog";
4. ${${$name}}="Monkey";
5. echo $name. "<br>";
6. echo ${$name}. "<br>";
7. echo $Cat. "<br>";
8. echo ${${$name}}. "<br>";
```

```
9. echo $Dog. "<br>";
10. ?>
```

Output:



In the above example, we have assigned a value to the variable name **Cat**. Value of reference variable **${$name}** is assigned as **Dog** and **${${$name}}** as **Monkey**.

Now we have printed the values as **$name, ${$name}, $Cat, ${${$name}}** and **$Dog.**

Expression in php

Almost everything in a PHP script is an expression. Anything that has a value is an expression. In a typical assignment statement ($x=100), a literal value, a function or operands processed by operators is an expression, anything that appears to the right of assignment operator (=)

Syntax

$x=100; //100 is an expression

$a=$b+$c; //b+$c is an expression

$c=add($a,$b); //add($a,$b) is an expresson

$val=sqrt(100); //sqrt(100) is an expression

$var=$x!=$y; //$x!=$y is an expression

expression with ++ and -- operators

These operators are called increment and decrement operators respectively. They are unary operators, needing just one operand and can be used in prefix or postfix manner, although with different effect on value of expression

Both prefix and postfix ++ operators increment value of operand by 1 (whereas -- operator decrements by 1). However, when used in assignment expression, prefix makesincremnt/decrement first and then followed by assignment. In case of postfix, assignment is done before increment/decrement

Uses postfix ++ operator

Example

```
<?php
$x=10;
$y=$x++; //equivalent to $y=$x followed by $x=$x+1
echo "x = $x y = $y";
?>
```

Output

This produces following result

x = 11 y = 10

Whereas following example uses prefix increment operator in assignment

Example

```
<?php
$x=10;
$y=++$x;; //equivalent to $x=$x+1 followed by $y=$x
echo "x = $x y = $y";
?>
```

Output

This produces following result

x = 11 y = 11

Expression with Ternary conditional operator

Ternary operator has three operands. First one is a logical expression. If it is TRU, second operand expression is evaluated otherwise third one is evaluated

Example

```
<?php
$marks=60;
$result= $marks<50 ? "fail" : "pass";
echo $result;
```

```
?>
```

Output

Following result will be displayed

pass

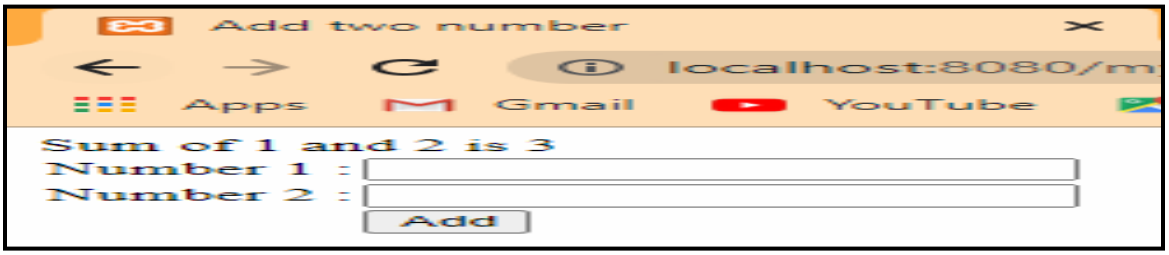
3.17 Basic Programming in PHP

Program 1:

```
<?php
if(isset($_POST['submit']))
{
$num1=$_POST['num1'];
$num2=$_POST['num2'];
$result=$num1+$num2;
echo "Sum of " .$num1. " and ".$num2. " is " .$result;
}
?>
<!DOCTYPE html>
<html>
<head>
<title>Add two number</title>
</head>
<body>
<table>
<form name="add" method="post">
<tr>
<td>Number 1 :</td>
<td><input type="text" name="num1" required></td>
</tr>
<tr>
<td>Number 2 :</td>
<td><input type="text" name="num2" required></td>
</tr>
```

```
<tr>
<td></td>
<td><input type="submit" value="Add" name="submit" /></td>
</tr>
</form>
</table>
</body>
</html>
```

Output:

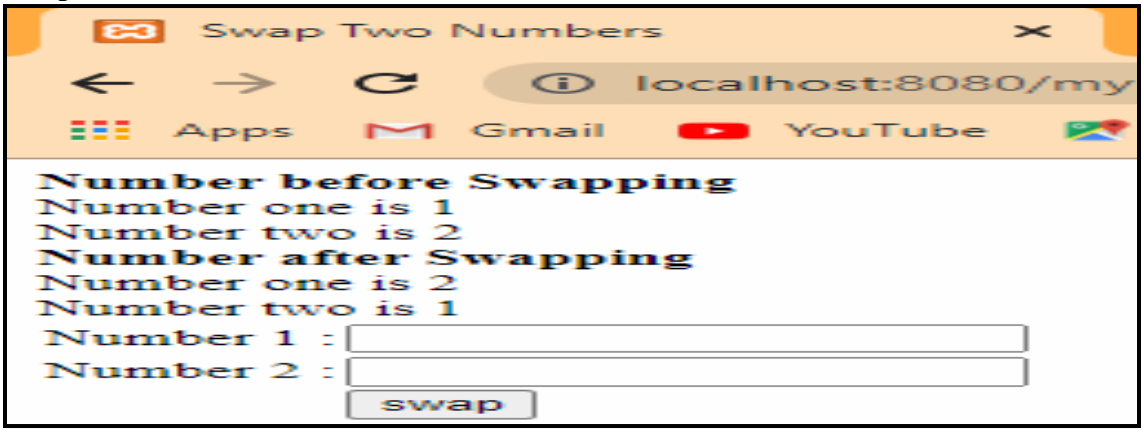


Program 2:

```
<?php
if(isset($_POST['submit']))
{
echo "<b>Number before Swapping</b><br>";
$num1=$_POST['num1'];
$num2=$_POST['num2'];
echo "Number one is $num1 <br />";
echo "Number two is $num2 <br />";
$temp=$num1;
$num1=$num2;
$num2=$temp;
echo "<b>Number after Swapping</b> <br />";
echo "Number one is $num1<br />";
echo "Number two is $num2<br />";
}
?>
<!DOCTYPE html>
<html>
```

```
<head>
<title>Swap Two Numbers</title>
</head>
<body>
<table>
<form name="swap" method="post">
<tr>
<td>Number 1 :</td>
<td><input type="text" name="num1" required></td>
</tr>
<tr>
<td>Number 2 :</td>
<td><input type="text" name="num2" required></td>
</tr>
<tr>
<td></td>
<td><input type="submit" value="swap" name="submit" /></td>
</tr>
</form>
</table>
</body>
</html>
```

Output:

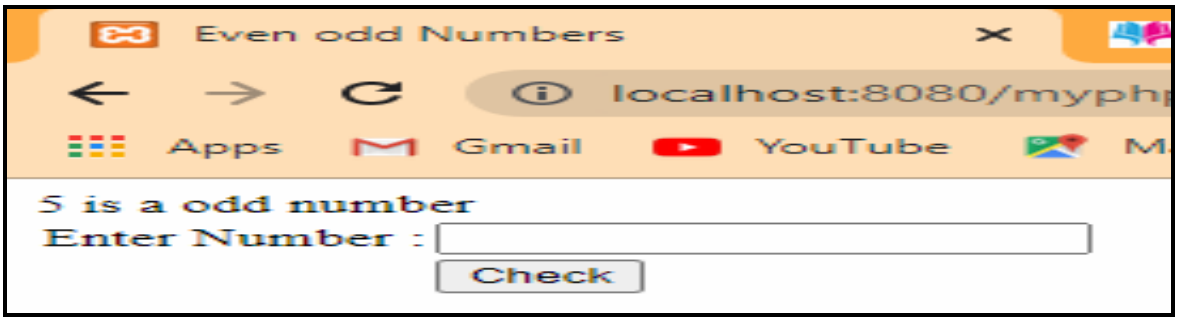


Program 3:

```
<?php
```

```
if(isset($_POST['submit']))
{
$num=$_POST['num'];
if($num%2==0){
  echo "$num  is a even number ";
}
else
{
echo "$num is a odd number";
}
}
?>
<!DOCTYPE html>
<html>
<head>
    <title>Even odd  Numbers</title>
</head>
<body>
<table>
<form name="evenodd" method="post">
<tr>
<td>Enter Number :</td>
<td><input type="text" name="num" required></td>
</tr>
<tr>
<td></td>
<td><input type="submit" value="Check" name="submit" /></td>
</tr>
</form>
</table>
</body>
</html>
```

Output:

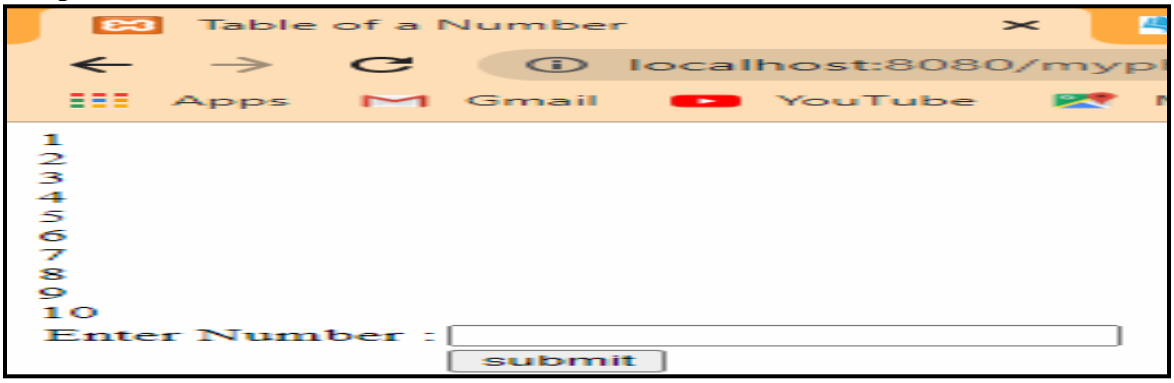


Program 4:

```
<?php
if(isset($_POST['submit']))
{
$num=$_POST['num'];
define('NUM',$num);
for($i=1 ; $i<=10 ; $i++)
{
echo $i*NUM;
echo '<br>';
}
}
?>
<!DOCTYPE html>
<html>
<head>
<title>Table of a Number</title>
</head>
<body>
<table>
<form name="table" method="post">
<tr>
<td>Enter Number :</td>
<td><input type="text" name="num" required></td>
</tr>
<tr>
<td></td>
<td><input type="submit" value="submit" name="submit" /></td>
```

```
</tr>
</form>
</table>
</body>
</html>
```

Output:

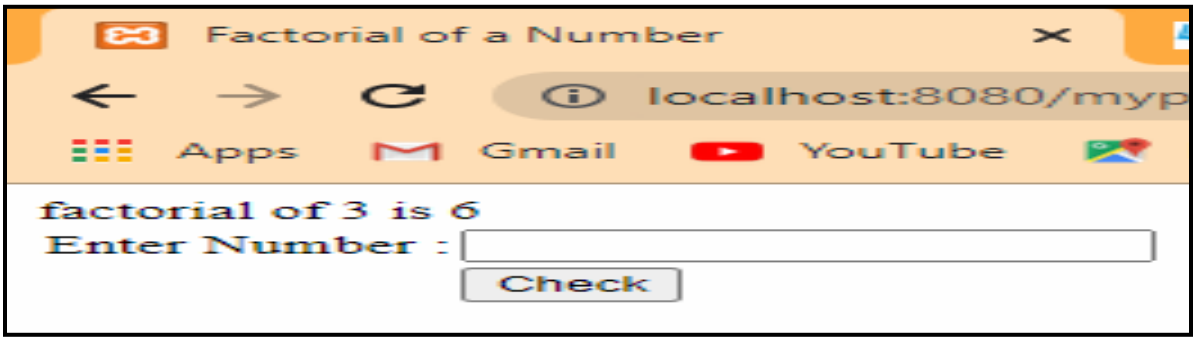


Program 5:

```
<?php
if(isset($_POST['submit']))
{
$num=$_POST['num'];
$fact = 1;
for ($i=$num; $i>=1; $i--)
{
 $fact = $fact * $i;
}
echo "factorial of  $num is  $fact";
}
?>
<!DOCTYPE html>
<html>
<head>
<title>Factorial of a Number</title>
</head>
<body>
<table>
```

```
<form name="factorial" method="post">
<tr>
<td>Enter Number :</td>
<td><input type="text" name="num" required></td>
</tr>
<tr>
<td></td>
<td><input type="submit" value="Check" name="submit" /></td>
</tr>
</form>
</table>
</body>
</html>
```

Output:

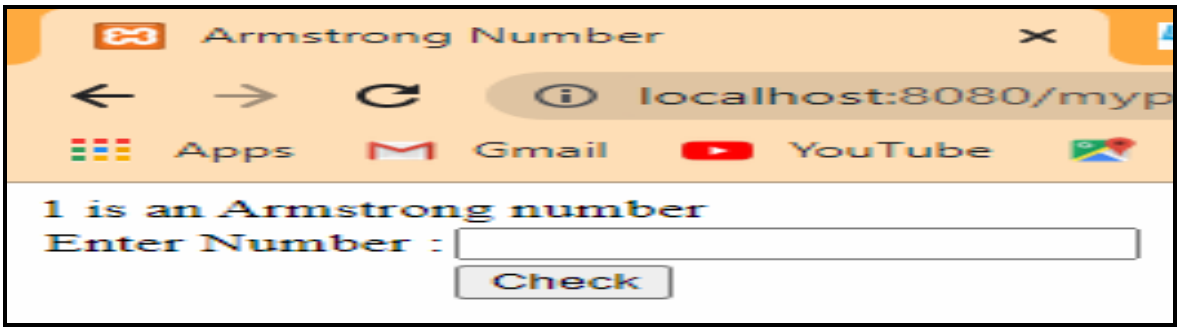


Program 6:

```
<?php
if(isset($_POST['submit']))
{
$num=$_POST['num'];
$sum=0;
$temp=$num;
while($temp!=0)
{
$rem=$temp%10;
$sum=$sum+$rem*$rem*$rem;
$temp=$temp/10;
}
if($num==$sum)
```

```
{
echo "$num is an Armstrong number";
}
else
{
echo "$num is not an Armstrong number";
}
}
?>
<!DOCTYPE html>
<html>
<head>
<title>Armstrong Number</title>
</head>
<body>
<table>
<form name="armstrong" method="post">
<tr>
<td>Enter Number :</td>
<td><input type="text" name="num" required></td>
</tr>
<tr>
<td></td>
<td><input type="submit" value="Check" name="submit" /></td>
</tr>
</form>
</table>
</body>
</html>
```

Output:

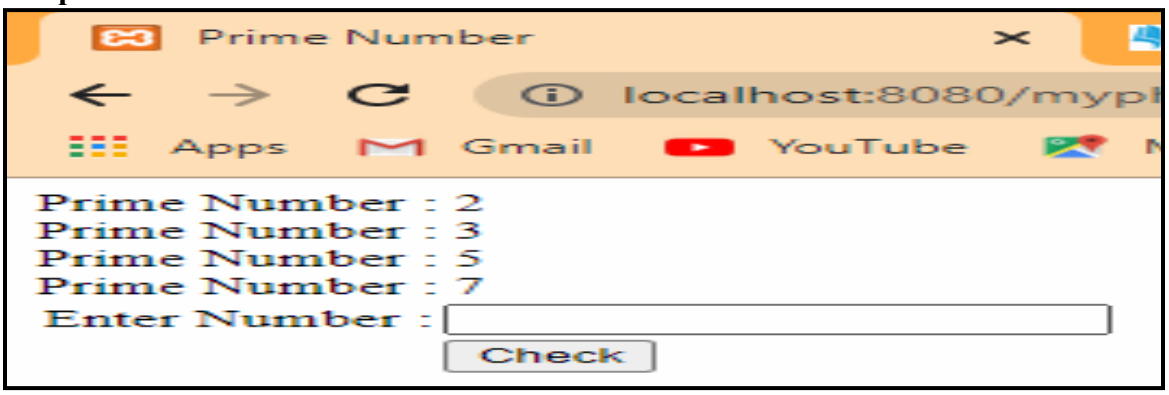


Program 7:

```
<?php
if(isset($_POST['submit']))
{
$num=$_POST['num'];
for($j=2;$j<=$num;$j++ )
{
for($k=2;$k<$j;$k++ )
{
if($j%$k==0)
{
break;
}
}
if( $k == $j )
echo "Prime Number : $j <br />";
}
}
?>
<!DOCTYPE html>
<html>
<head>
<title>Prime Number</title>
</head>
<body>
```

```
<table>
<form name="prime" method="post">
<tr>
<td>Enter Number :</td>
<td><input type="text" name="num" required></td>
</tr>
<tr>
<td></td>
<td><input type="submit" value="Check" name="submit" /></td>
</tr>
</form>
</table>
</body>
</html>
```

Output:



3.18 Operators (Arithmetic, logical, comparison, operator precedence)

PHP Operators

PHP Operator is a symbol i.e used to perform operations on operands. In simple words, operators are used to perform operations on variables or values.

For example:

$num=10+20;//+ is the operator and 10,20 are operands

In the above example, + is the binary + operator, 10 and 20 are operands and $num is variable.

PHP Operators can be categorized in following forms:

- **Arithmetic Operators**
- **Assignment Operators**
- **Bitwise Operators**
- **Comparison Operators**
- **Incrementing/Decrementing Operators**
- **Logical Operators**

- **String Operators**
- **Array Operators**
- **Type Operators**
- **Execution Operators**
- **Error Control Operators**

We can also categorize operators on behalf of operands. They can be categorized in 3 forms:

- **Unary Operators:** works on single operands such as ++, -- etc.
- **Binary Operators:** works on two operands such as binary +, -, *, / etc.
- **Ternary Operators:** works on three operands such as "?:".

1.Arithmetic Operators

The PHP arithmetic operators are used to perform common arithmetic operations such as addition, subtraction, etc. with numeric values.

Operator	Name	Example	Explanation
+	Addition	$x+ $y	Sum of operands
-	Subtraction	$x - $y	Difference of operands
*	Multiplication	$x * $y	Product of operands
/	Division	$x / $y	Quotient of operands
%	Modulus	$x % $y	Remainder of operands
**	Exponentiation	$x ** $y	$x raised to the power $y

The exponentiation (**) operator has been introduced in PHP 5.6.

2.Assignment Operators

The assignment operators are used to assign value to different variables. The basic assignment operator is "=".

Operator	Name	Example	Explanation
=	Assign	$x = $y	The value of right operand is assigned to the left operand.
+=	Add then Assign	$x+= $y	Addition same as $x = $x + $y
-=	Subtract then Assign	$x -= $y	Subtraction same as $x = $x - $y
*=	Multiply then Assign	$x *= $y	Multiplication same as $x = $x * $y
/=	Divide then Assign (quotient)	$x /= $y	Find quotient same as $x= $x / $y

%=	Divide then Assign (remainder)	$x%= $y	Find remainder same as $x = $x % $y

3.Bitwise Operators

The bitwise operators are used to perform bit-level operations on operands. These operators allow the evaluation and manipulation of specific bits within the integer.

Operator	Name	Example	Explanation
&	And	$x & $y	Bits that are 1 in both $x and $y are set to 1, otherwise 0.
|	Or (Inclusive or)	$x | $y	Bits that are 1 in either $x or $y are set to 1
^	Xor (Exclusive or)	$x^ $y	Bits that are 1 in either $x or $y are set to 0.
~	Not	~$x	Bits that are 1 set to 0 and bits that are 0 are set to 1
<<	Shift left	$x<< $y	Left shift the bits of operand $x $y steps
>>	Shift right	$x>> $y	Right shift the bits of $x operand by $y number of places

4.Comparison Operators

Comparison operators allow comparing two values, such as number or string. Below the list of comparison operators are given:

Operator	Name	Example	Explanation
==	Equal	$a == $b	Return TRUE if $a is equal to $b
===	Identical	$a === $b	Return TRUE if $a is equal to $b, and they are of same data type
!==	Not identical	$a !== $b	Return TRUE if $a is not equal to $b, and they are not of same data type
!=	Not equal	$a != $b	Return TRUE if $a is not equal to $b
<>	Not equal	$a <> $b	Return TRUE if $a is not equal to $b
<	Less than	$a < $b	Return TRUE if $a is less than $b
>	Greater than	$a > $b	Return TRUE if $a is

			greater than $b
<=	Less than or equal to	$a <= $b	Return TRUE if $a is less than or equal $b
>=	Greater than or equal to	$a >= $b	Return TRUE if $a is greater than or equal $b
<=>	Spaceship	$a <=>$b	Return -1 if $a is less than $b Return 0 if $a is equal $b Return 1 if $a is greater than $b

5.Incrementing/Decrementing Operators

The increment and decrement operators are used to increase and decrease the value of a variable.

Operator	Name	Example	Explanation
++	Increment	++$a	Increment the value of $a by one, then return $a
		$a++	Return $a, then increment the value of $a by one
--	decrement	--$a	Decrement the value of $a by one, then return $a
		$a--	Return $a, then decrement the value of $a by one

6. Logical Operators

The logical operators are used to perform bit-level operations on operands. These operators allow the evaluation and manipulation of specific bits within the integer.

Operator	Name	Example	Explanation
and	And	$a and $b	Return TRUE if both $a and $b are true
Or	Or	$a or $b	Return TRUE if either $a or $b is true
xor	Xor	$a xor $b	Return TRUE if either $ or $b is true but not both
!	Not	! $a	Return TRUE if $a is not true

&&	And	$a && $b	Return TRUE if either $a and $b are true
||	Or	$a || $b	Return TRUE if either $a or $b is true

7.String Operators

The string operators are used to perform the operation on strings. There are two string operators in PHP, which are given below:

Operator	Name	Example	Explanation
.	Concatenation	$a . $b	Concatenate both $a and $b
.=	Concatenation and Assignment	$a .= $b	First concatenate $a and $b, then assign the concatenated string to $a, e.g. $a = $a . $b

8. Array Operators

The array operators are used in case of array. Basically, these operators are used to compare the values of arrays.

Operator	Name	Example	Explanation
+	Union	$a + $y	Union of $a and $b
==	Equality	$a == $b	Return TRUE if $a and $b have same key/value pair
!=	Inequality	$a != $b	Return TRUE if $a is not equal to $b
===	Identity	$a === $b	Return TRUE if $a and $b have same key/value pair of same type in same order
!==	Non-Identity	$a !== $b	Return TRUE if $a is not identical to $b
<>	Inequality	$a <> $b	Return TRUE if $a is not equal to $b

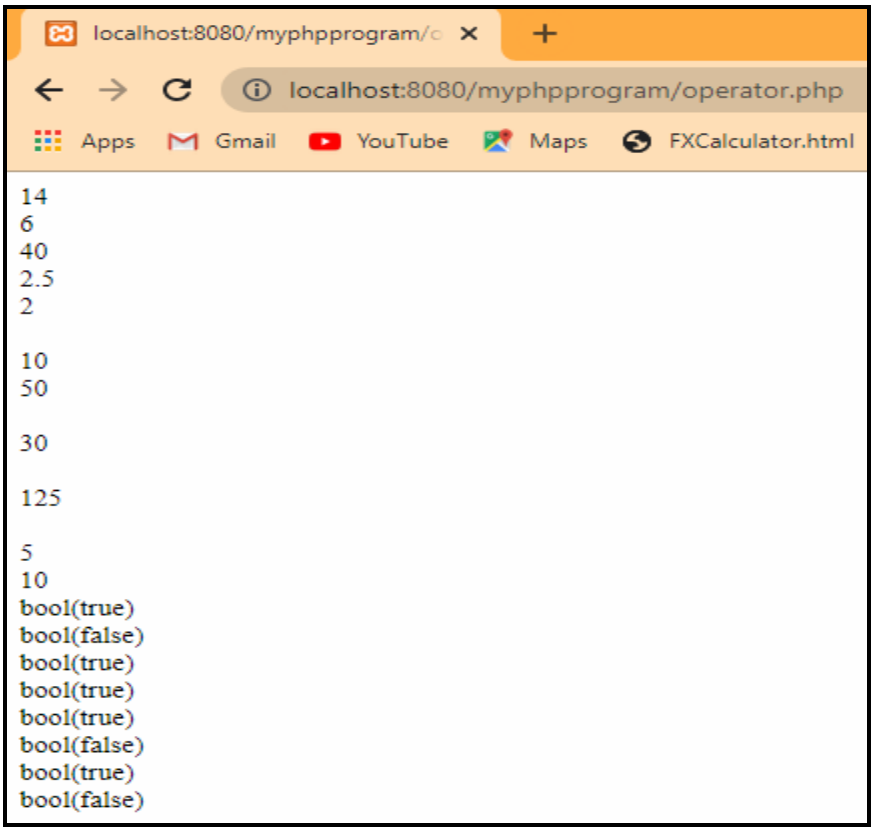
Operator program 1:

```
<?php
$x = 10;
$y = 4;
echo($x + $y); // Outputs: 14
echo "</br>";
echo($x - $y); // Outputs: 6
```

```
echo "</br>";
echo($x * $y); // Outputs: 40
echo "</br>";
echo($x / $y); // Outputs: 2.5
echo "</br>";
echo($x % $y); // Outputs: 2
echo "</br>";
$x = 10;
echo "</br>";
echo $x; // Outputs: 10
$x = 20;
echo "</br>";
$x += 30;
echo $x; // Outputs: 50
echo "</br>";
$x = 50;
$x -= 20;
echo "</br>";
echo $x; // Outputs: 30
echo "</br>";
$x = 5;
$x *= 25;
echo "</br>";
echo $x; // Outputs: 125
$x = 50;
echo "</br>";
$x /= 10;
echo "</br>";
echo $x; // Outputs: 5
echo "</br>";
$x = 100;
$x %= 15;
```

```
echo $x; // Outputs: 10
$x = 25;
$y = 35;
$z = "25";
echo "</br>";
var_dump($x == $z);  // Outputs: boolean true
echo "</br>";
var_dump($x === $z); // Outputs: boolean false
echo "</br>";
var_dump($x != $y);  // Outputs: boolean true
echo "</br>";
var_dump($x !== $z); // Outputs: boolean true
echo "</br>";
var_dump($x < $y);   // Outputs: boolean true
echo "</br>";
var_dump($x > $y);   // Outputs: boolean false
echo "</br>";
var_dump($x <= $y);  // Outputs: boolean true
echo "</br>";
var_dump($x >= $y);  // Outputs: boolean false
?>
```

Output:



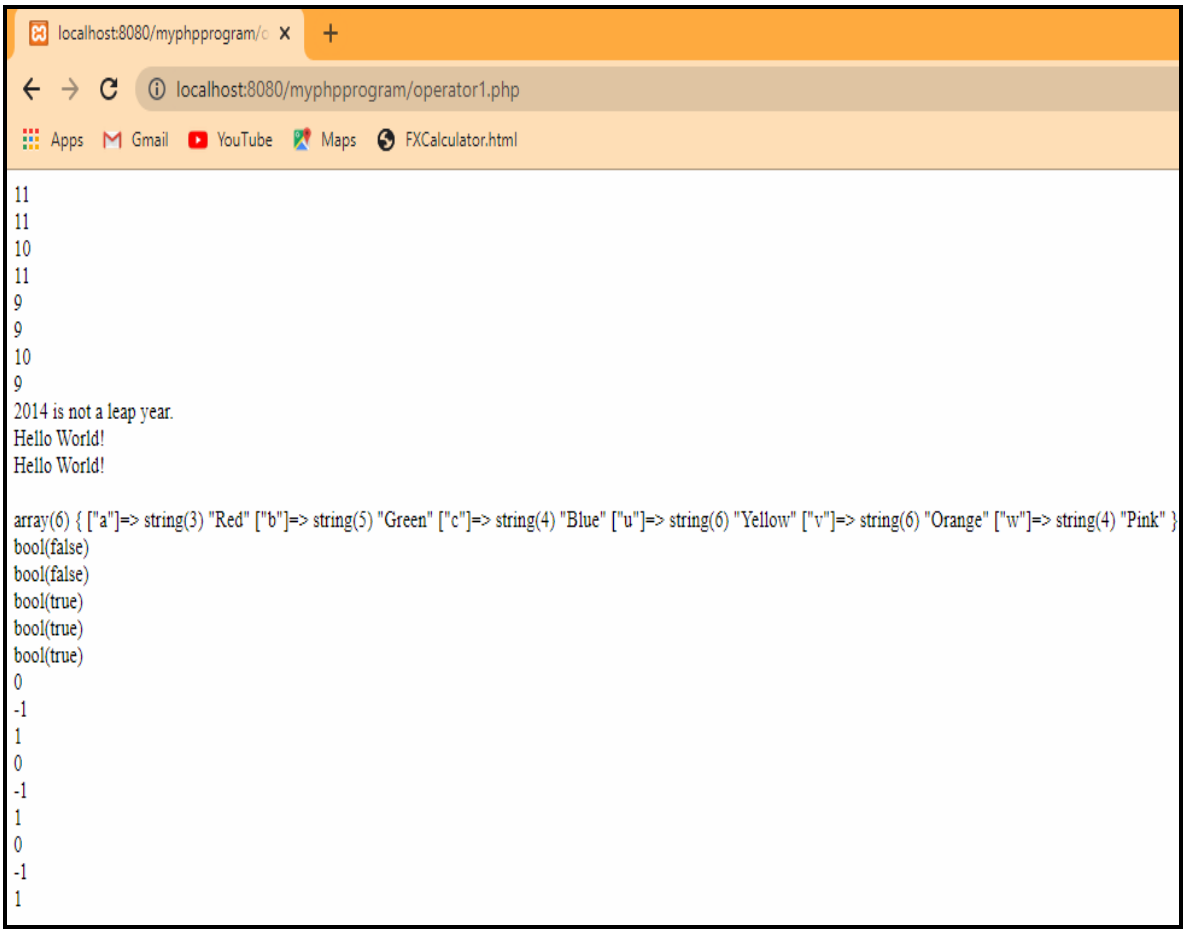
Operator program 2:

```
<?php
$x = 10;
echo ++$x; // Outputs: 11
echo "</br>";
echo $x;   // Outputs: 11
echo "</br>";
$x = 10;
echo $x++; // Outputs: 10
echo "</br>";
echo $x;   // Outputs: 11
echo "</br>";
$x = 10;
echo --$x; // Outputs: 9
echo "</br>";
echo $x;   // Outputs: 9
```

```
echo "</br>";
$x = 10;
echo $x--; // Outputs: 10
echo "</br>";
echo $x;   // Outputs: 9
echo "</br>";
$year = 2014;
// Leap years are divisible by 400 or by 4 but not 100
if(($year % 400 == 0) || (($year % 100 != 0) && ($year % 4 == 0))){
    echo "$year is a leap year.";
} else{
    echo "$year is not a leap year.";
}
echo "</br>";
$x = "Hello";
$y = " World!";
echo $x . $y; // Outputs: Hello World!
$x .= $y;
echo "</br>";
echo $x; // Outputs: Hello World!
$x = array("a" => "Red", "b" => "Green", "c" => "Blue");
echo "</br>";
$y = array("u" => "Yellow", "v" => "Orange", "w" => "Pink");
echo "</br>";
$z = $x + $y; // Union of $x and $y
var_dump($z);
var_dump($x == $y);   // Outputs: boolean false
echo "</br>";
var_dump($x === $y);  // Outputs: boolean false
echo "</br>";
var_dump($x != $y);   // Outputs: boolean true
echo "</br>";
```

```
var_dump($x <> $y);   // Outputs: boolean true
echo "</br>";
var_dump($x !== $y);  // Outputs: boolean true
// Comparing Integers
echo "</br>";
echo 1 <=> 1; // Outputs: 0
echo "</br>";
echo 1 <=> 2; // Outputs: -1
echo "</br>";
echo 2 <=> 1; // Outputs: 1
echo "</br>";
// Comparing Floats
echo 1.5 <=> 1.5; // Outputs: 0
echo "</br>";
echo 1.5 <=> 2.5; // Outputs: -1
echo "</br>";
echo 2.5 <=> 1.5; // Outputs: 1
echo "</br>";
// Comparing Strings
echo "x" <=> "x"; // Outputs: 0
echo "</br>";
echo "x" <=> "y"; // Outputs: -1
echo "</br>";
echo "y" <=> "x"; // Outputs: 1
?>
```

Output:



Program2:

```php
<?php
$x = 10;
echo ++$x; // Outputs: 11
echo "</br>";
echo $x;   // Outputs: 11
echo "</br>";
$x = 10;
echo $x++; // Outputs: 10
echo "</br>";
echo $x;   // Outputs: 11
echo "</br>";
$x = 10;
echo --$x; // Outputs: 9
echo "</br>";
echo $x;   // Outputs: 9
echo "</br>";
$x = 10;
echo $x--; // Outputs: 10
echo "</br>";
```

```
echo $x;   // Outputs: 9
echo "</br>";
$year = 2014;
// Leap years are divisible by 400 or by 4 but not 100
if(($year % 400 == 0) || (($year % 100 != 0) && ($year % 4 == 0))){
    echo "$year is a leap year.";
} else{
    echo "$year is not a leap year.";
}
echo "</br>";
$x = "Hello";
$y = " World!";
echo $x . $y; // Outputs: Hello World!
$x .= $y;
echo "</br>";
echo $x; // Outputs: Hello World!
$x = array("a" => "Red", "b" => "Green", "c" => "Blue");
echo "</br>";
$y = array("u" => "Yellow", "v" => "Orange", "w" => "Pink");
echo "</br>";
$z = $x + $y; // Union of $x and $y
var_dump($z);
print_r($z);

var_dump($x == $y);   // Outputs: boolean false
$a=10;
$b="10";
echo $a==$b ;
echo "\n";
echo "</br>";
var_dump($x === $y);  // Outputs: boolean false
echo "</br>";
var_dump($x != $y);   // Outputs: boolean true
echo "</br>";
var_dump($x <> $y);   // Outputs: boolean true
echo "</br>";
var_dump($x !== $y);  // Outputs: boolean true
// Comparing Integers
echo "</br>";
echo 1 <=> 1; // Outputs: 0
echo "</br>";
echo 1 <=> 2; // Outputs: -1
echo "</br>";
echo 2 <=> 1; // Outputs: 1
echo "</br>";
```

```
// Comparing Floats
echo 1.5 <=> 1.5; // Outputs: 0
echo "</br>";
echo 1.5 <=> 2.5; // Outputs: -1
echo "</br>";
echo 2.5 <=> 1.5; // Outputs: 1
echo "</br>";
// Comparing Strings
echo "x" <=> "x"; // Outputs: 0
echo "</br>";
echo "x" <=> "y"; // Outputs: -1
echo "</br>";
echo "y" <=> "x"; // Outputs: 1
// if empty($user) = TRUE, set $status = "anonymous"
echo $status = (empty($user)) ? "anonymous" : "logged in";
echo("<br>");

$user = "John Doe";
// if empty($user) = FALSE, set $status = "logged in"
echo $status = (empty($user)) ? "anonymous" : "logged in";
$x = 50;
$y = 30;
  if ($x == 50 and $y == 30)
      echo "and Success \n";

  if ($x == 50 or $y == 20)
      echo "or Success \n";

  if ($x == 50 xor $y == 20)
      echo "xor Success \n";

  if ($x == 50 && $y == 30)
      echo "&& Success \n";

  if ($x == 50 || $y == 20)
      echo "|| Success \n";

  if (!$z)
      echo "! Success \n";
?>
```

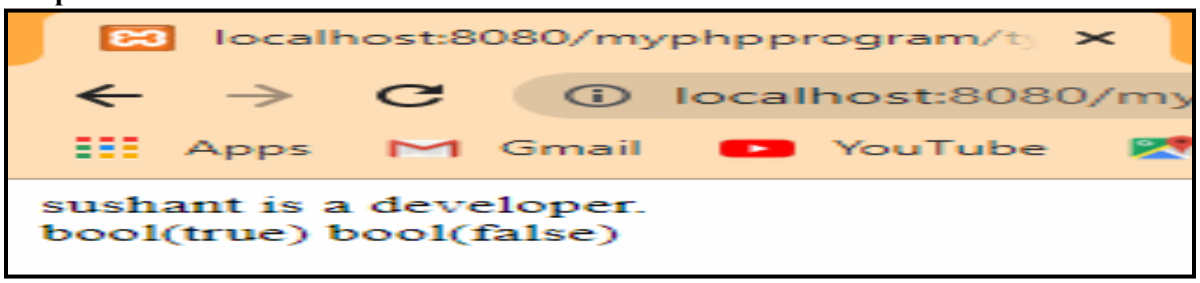
9.Type Operators

The type operator **instanceof** is used to determine whether an object, its parent and its derived class are the same type or not. Basically, this operator determines which certain class the object belongs to. It is used in object-oriented programming.

```
<?php
   //class declaration
   class Developer
   {}
   class Programmer
   {}
   //creating an object of type Developer
   $sushant = new Developer();

   //testing the type f object
   if( $sushant instanceof Developer)    {
      echo "sushant   is a developer.";    }
   else     {
      echo "sushant  is a programmer.";     }
   echo "</br>";
   var_dump($sushant instanceof Developer);          //It will return true.
   var_dump($sushant instanceof Programmer);       //It will return false.
?>
```

Output:



10.PHP Operators Precedence

Let's see the precedence of PHP operators with associativity.

Operators	Additional Information	Associativity
clone new	clone and new	non-associative
[	array()	left
**	Arithmetic	right

++ -- ~ (int) (float) (string) (array) (object) (bool) @	increment/decrement and types	right
instanceof	Types	non-associative
!	logical (negation)	right
* / %	Arithmetic	left
+ - .	arithmetic and string concatenation	left
<< >>	bitwise (shift)	left
< <= > >=	Comparison	non-associative
== != === !== <>	Comparison	non-associative
&	bitwise AND	left
^	bitwise XOR	left
|	bitwise OR	left
&&	logical AND	left
||	logical OR	left
?:	ternary	left
= += -= *= **= /= .= %= &= |= ^= <<= >>= =>	assignment	right
and	logical	left
xor	logical	left
Or	logical	left
,	many uses (comma)	left

PHP Variables

In PHP, a variable is declared using a **$ sign** followed by the variable name. Here, some important points to know about variables:

- As PHP is a loosely typed language, so we do not need to declare the data types of the variables. It automatically analyzes the values and makes conversions to its correct datatype.
- After declaring a variable, it can be reused throughout the code.
- Assignment Operator (=) is used to assign the value to a variable.

Syntax of declaring a variable in PHP is given below:

```
$variablename=value;
```

Rules for declaring PHP variable:

- A variable must start with a dollar ($) sign, followed by the variable name.
- It can only contain alpha-numeric character and underscore (A-z, 0-9, _).
- A variable name must start with a letter or underscore (_) character.
- A PHP variable name cannot contain spaces.
- One thing to be kept in mind that the variable name cannot start with a number or special symbols.
- PHP variables are case-sensitive, so $name and $NAME both are treated as different variable.

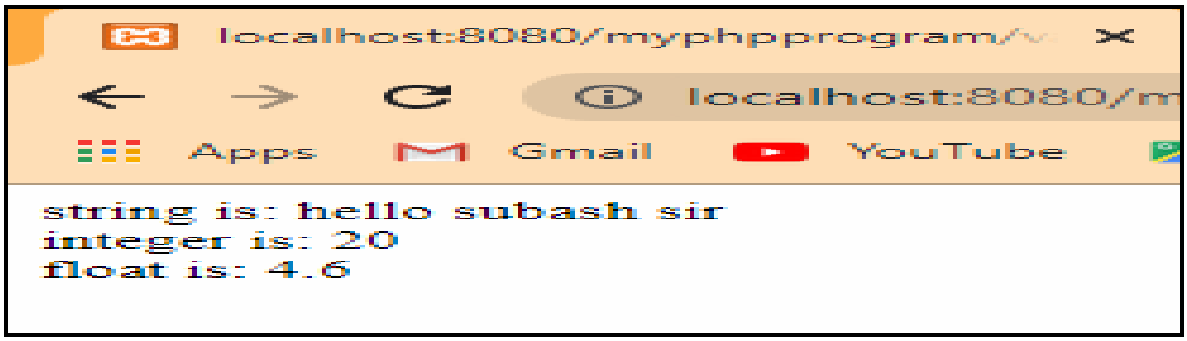
PHP Variable: Declaring string, integer, and float

Let's see the example to store string, integer, and float values in PHP variables.

File: variable.php

```
<?php
   $str="hello subash sir";
   $x=20;
   $y=4.6;
   echo "string is: $str <br/>";
   echo "integer is: $x <br/>";
   echo "float is: $y <br/>";
   ?>
```

Output:



PHP | Defining Constants

In a production level code it is very important to keep information as either variables or constants rather than using them explicitly. A PHP constant is nothing but an identifier for a simple value that tends not to change over time(such as the domain name of a website eg. www.facebook.org). It is ideal to keep all the constants in a single PHP script so the maintenance is made easier. A valid constant name must start with an alphabet or underscore and requires no '$'. It is to be noted, constants are irrespective of their scope i.e constants are automatically of global scope.

In order to create a constant in PHP, we must use the define() method.

Syntax:

bool define(identifier, value, case-insensitivity)

Parameters: The function has two required parameters and one optional parameter.

identifier: Specifies the name to be assigned to the constant.

value: Specifies the value to be assigned to the constant.

case-insensitivity(Optional): Specifies whether the constant identifier should be case-insensitive. By default it is set to false i.e. case-sensitive.

Return Type: This method returns TRUE on success and FALSE on Failure.

Below program illustrates defining case-insensitive constants:

```
<?php

  // case-insensitive constants
  define("Constant","Hello Geeks!",TRUE);
  echo constant;
  echo Constant;
?>
```

PHP Control Structures

PHP If Statement

The if construct allows you to execute a piece of code if the expression provided along with it evaluates to true.

Let's have a look at the following example to understand how it actually works.

```
1    <?php
2    $age = 50;
```

```
3
4    if ($age > 30)
5    {
6      echo "Your age is greater than 30!";
7    }
8    ?>
```

The above example should output the Your age is greater than 30! message since the expression evaluates to true. In fact, if you want to execute only a single statement, the above example can be rewritten without brackets, as shown in the following snippet.

```
1    <?php
2    $age = 50;
3
4    if ($age > 30)
5      echo "Your age is greater than 30!";
6    ?>
```

On the other hand, if you have more than one statement to execute, you must use brackets, as shown in the following snippet.

```
1    <?php
2    if (is_array($user))
3    {
4      $user_id = isset($user['user_id']) ? $user['user_id'] : '';
5      $username = isset($user['username']) ? $user['username'] : '';
6      // and more statements...
7    }
8    ?>
```

PHP Else Statement

In the previous section, we discussed the if construct, which allows you to execute a piece of code if the expression evaluates to true. On the other hand, if the expression evaluates to false, it won't do anything. More often than not, you also want to execute a different code snippet if the expression evaluates to false. That's where the else statement comes into the picture.

You always use the else statement in conjunction with an if statement. Basically, you can define it as shown in the following pseudo-code.

```
1    if (expression)
2    {
3      // code is executed if the expression evaluates to TRUE
4    }
5    else
6    {
7      // code is executed if the expression evaluates to FALSE
8    }
```

Let's revise the previous example to understand how it works.

```
<?php
$age = 50;

if ($age < 30)
{
  echo "Your age is less than 30!";
}
else
{
  echo "Your age is greater than or equal to 30!";
}
?>
```

So when you have two choices, and one of them must be executed, you can use the if-else construct.

PHP Else If Statement

We can consider the elseif statement as an extension to the if-else construct. If you've got more than two choices to choose from, you can use the elseif statement.

Let's study the basic structure of the elseif statement, as shown in the following pseudo-code.

```
if (expression1)
{
  // code is executed if the expression1 evaluates to TRUE
}
elseif (expression2)
{
  // code is executed if the expression2 evaluates to TRUE
}
elseif (expression3)
{
  // code is executed if the expression3 evaluates to TRUE
}
else
{
  // code is executed if the expression1, expression2 and expression3 evaluates to FALSE, a default choice
}
```

Again, let's try to understand it using a real-world example.

```
<?php
$age = 50;

```

```
if ($age < 30)
{
  echo "Your age is less than 30!";
}
elseif ($age > 30 && $age < 40)
{
  echo "Your age is between 30 and 40!";
}
elseif ($age > 40 && $age < 50)
{
  echo "Your age is between 40 and 50!";
}
else
{
  echo "Your age is greater than 50!";
}
?>
```

.

PHP Switch Statement

The switch statement is somewhat similar to the elseif statement which we've just discussed in the previous section. The only difference is the expression which is being checked.

In the case of the elseif statement, you have a set of different conditions, and an appropriate action will be executed based on a condition. On the other hand, if you want to compare a variable with different values, you can use the switch statement.

As usual, an example is the best way to understand the switch statement.

```
<?php
$favourite_site = 'Code';

switch ($favourite_site) {
  case 'Business':
    echo "My favourite site is business.tutsplus.com!";
    break;
  case 'Code':
    echo "My favourite site is code.tutsplus.com!";
    break;
  case 'Web Design':
    echo "My favourite site is webdesign.tutsplus.com!";
    break;
  case 'Music':
    echo "My favourite site is music.tutsplus.com!";
```

```
    break;
  case 'Photography':
    echo "My favourite site is photography.tutsplus.com!";
    break;
  default:
    echo "I like everything at tutsplus.com!";
}
?>
```

As you can see in the above example, we want to check the value of the `$favourite_site` variable, and based on the value of the `$favourite_site` variable, we want to print a message.

For each value you want to check with the `$favourite_site` variable, you have to define the `case` block. If the value is matched with a case, the code associated with that case block will be executed. After that, you need to use the `break` statement to end code execution. If you don't use the `break` statement, script execution will be continued up to the last block in the switch statement.

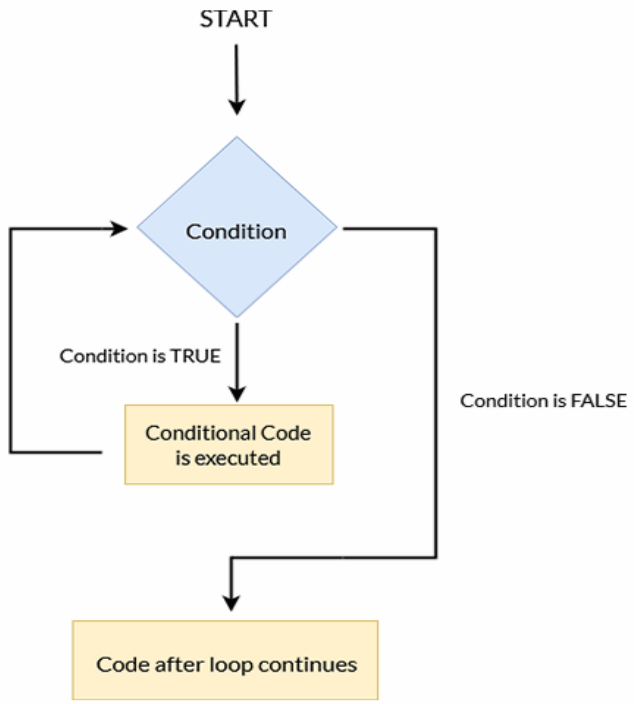
Finally, if you want to execute a piece of code if the variable's value doesn't match any case, you can define it under the `default` block. Of course, it's not mandatory—it's just a way to provide a *default* case.

So that's the story of conditional control structures. We'll discuss loops in PHP in the next section.

Loops in PHP

Loops in PHP are useful when you want to execute a piece of code repeatedly until a condition evaluates to false. So code is executed repeatedly as long as a condition evaluates to true, and as soon as the condition evaluates to false, the script continues executing the code after the loop.

The following flowchart explains how loops work in PHP.



As you can see in the above screenshot, a loop contains a condition. If the condition evaluates to true, the conditional code is executed. After execution of the conditional code, control goes back to the loop condition, and the flow continues until the condition evaluates to false.
In this section, we'll go through the different types of loops supported in PHP.

While Loop in PHP

The while loop is used when you want to execute a piece of code repeatedly until the while condition evaluates to false.
You can define it as shown in the following pseudo-code.

```
while (expression)
{
  // code to execute as long as expression evaluates to TRUE
}
```

Let's have a look at a real-world example to understand how the while loop works in PHP.

```
<?php
$max = 0;
echo $i = 0;
echo ",";
echo $j = 1;
echo ",";
$result=0;

```

```
while ($max < 10 )
{
   $result = $i + $j;

   $i = $j;
   $j = $result;

   $max = $max + 1;
   echo $result;
   echo ",";
}
?>
```

If you're familiar with the Fibonacci series, you might recognize what the above program does—it outputs the Fibonacci series for the first ten numbers. The while loop is generally used when you don't know the number of iterations that are going to take place in a loop.

Do-While Loop in PHP

The do-while loop is very similar to the while loop, with the only difference being that the while condition is checked at the end of the first iteration. Thus, we can guarantee that the loop code is executed at least once, irrespective of the result of the while expression.

Let's have a look at the syntax of the do-while loop.

```
do
{
  // code to execute
} while (expression);
```

Let's go through a real-world to understand possible cases where you can use the do-while loop.

```
<?php
$handle = fopen("file.txt", "r");
if ($handle)
{
   do
   {
      $line = fgets($handle);

      // process the line content

   } while($line !== false);
}
fclose($handle);
?>
```

In the above example, we're trying to read a file line by line. Firstly, we've opened a file for reading. In our case, we're not sure if the file contains any content at all. Thus, we need to execute the fgets function at least once to check if a file contains any content. So we can use the do-while loop here. do-while evaluates the condition *after* the first iteration of the loop.

For Loop in PHP

Generally, the for loop is used to execute a piece of code a specific number of times. In other words, if you already know the number of times you want to execute a block of code, it's the for loop which is the best choice.

Let's have a look at the syntax of the for loop.

```
for (expr1; expr2; expr3)
{
  // code to execute
}
```

The expr1 expression is used to initialize variables, and it's always executed.

The expr2 expression is also executed at the beginning of a loop, and if it evaluates to true, the loop code is executed. After execution of the loop code, the expr3 is executed. Generally, the expr3 is used to alter the value of a variable which is used in the expr2 expression.

Let's go through the following example to see how it works.

```
<?php
for ($i=1; $i<=10; ++$i)
{
  echo sprintf("The square of %d is %d.</br>", $i, $i*$i);
}
?>
```

The above program outputs the square of the first ten numbers. It initializes $i to 1, repeats as long as $i is less than or equal to 10, and adds 1 to $i at each iteration.

For Each in PHP

The foreach loop is used to iterate over array variables. If you have an array variable, and you want to go through each element of that array, the foreach loop is the best choice.

Let's have a look at a couple of examples.

```
<?php
$fruits = array('apple', 'banana', 'orange', 'grapes');
foreach ($fruits as $fruit)
{
  echo $fruit;
  echo "<br/>";
}

$employee = array('name' => 'John Smith', 'age' => 30, 'profession' => 'Software Engineer');
foreach ($employee as $key => $value)
```

```
{
  echo sprintf("%s: %s</br>", $key, $value);
  echo "<br/>";
}
?>
```

If you want to access array values, you can use the first version of the foreach loop, as shown in the above example. On the other hand, if you want to access both a key and a value, you can do it as shown in the $employee example above.

Breaking Out of the Loop

There are times when you might want to break out of a loop before it runs its course. This can be achieved easily using the break keyword. It will get you out of the current for, foreach, while, do-while, or switch structure.

You can also use break to get out of multiple nested loops by supplying a numeric argument. For example, using break 3 will break you out of 3 nested loops. However, you cannot pass a variable as the numeric argument if you are using a PHP version greater than or equal to 5.4.

```
<?php

echo 'Simple Break';
for($i = 1; $i <= 2; $i++) {
    echo "\n".'$i = '.$i.' ';
    for($j = 1; $j <= 5; $j++) {
        if($j == 2) {
            break;
        }
        echo '$j = '.$j.' ';
    }
}

/*
Simple Break
i = 1 j = 1
i = 2 j = 1
*/

echo 'Multi-level Break';
for($i = 1; $i <= 2; $i++) {
    echo "\n".'$i = '.$i.' ';
    for($j = 1; $j <= 5; $j++) {
        if($j == 2) {
            break 2;
```

```
26          }
27          echo '$j = '.$j.' ';
28      }
29  }
30
31  /*
32  Multi-level Break
33  i = 1 j = 1
34  */
35
36  ?>
```

Another keyword that can interrupt loops in PHP is continue. However, this only skips the rest of the current loop iteration instead of breaking out of the loop altogether. Just like break, you can also use a numerical value with continue to specify how many nested loops it should skip for the current iteration.

```
01  <?php
02
03  echo 'Simple Continue';
04  for($i = 1; $i <= 2; $i++) {
05      echo "\n".'$i = '.$i.' ';
06      for($j = 1; $j <= 5; $j++) {
07          if($j == 2) {
08              continue;
09          }
10          echo '$j = '.$j.' ';
11      }
12  }
13
14  /*
15  Simple Continue
16  i = 1 j = 1 j = 3 j = 4 j = 5
17  i = 2 j = 1 j = 3 j = 4 j = 5
18  */
19
20  echo 'Multi-level Continue';
21  for($i = 1; $i <= 2; $i++) {
22      echo "\n".'$i = '.$i.' ';
23      for($j = 1; $j <= 5; $j++) {
24          if($j == 2) {
25              continue 2;
```

```
        }
        echo '$j = '.$j.' ';
    }
}

/*
Multi-level Continue
i = 1 j = 1
i = 2 j = 1
*/

?>
```

goto

The goto statement allows execution to "jump" to another place in the program. You specify execution points by adding a label, which is an identifier followed by a colon (:). You then jump to the label from another location in the script via the goto statement:

```
for ($i = 0; $i < $count; $i++) {
// oops, found an error
if ($error) {
goto cleanup;
}
}
cleanup:
// do some cleanup
```

PHP include and require Statements

It is possible to insert the content of one PHP file into another PHP file (before the server executes it), with the include or require statement.

The include and require statements are identical, except upon failure:

a.require will produce a fatal error (E_COMPILE_ERROR) and stop the script

b.include will only produce a warning (E_WARNING) and the script will continue

So, if you want the execution to go on and show users the output, even if the include file is missing, use the include statement. Otherwise, in case of FrameWork, CMS, or a complex PHP application coding, always use the require statement to include a key file to the flow of execution. This will help avoid compromising your application's security and integrity, just in-case one key file is accidentally missing.

Including files saves a lot of work. This means that you can create a standard header, footer, or menu file for all your web pages. Then, when the header needs to be updated, you can only update the header include file.

Syntax

include '*filename*';

or

require '*filename*';

PHP include Examples
Example 1
Assume we have a standard footer file called "footer.php", that looks like this:

```
<?php
echo "<p>Copyright &copy; 1999-" . date("Y") . "ss.com</p>";
?>
```

To include the footer file in a page, use the include statement:
Example

```
<html>
<body>

<h1>Welcome to my home page!</h1>
<p>Some text.</p>
<p>Some more text.</p>
<?php include 'footer.php';?>

</body>
</html>
```

Embedding PHP in Web Pages

We can use PHP in HTML code by simply adding a PHP tag without doing any extra work.
Example : First open the file in any editor and then write HTML code according to requirement. If we have to add PHP code then we can add by simply adding **<?php ….. ?>** tags in between and add your PHP code accordingly.
PHP

```
<!DOCTYPE html>
<html>
  <head>
    <title>PHP</title>
  </head>
  <body>
     <h1>
       <?php
            echo "hii GeeksforGeeks "
         ?>
```

```
    </h1>
  </body>
</html>
```

THE END